Symbiosis Institute of Technology, Pune

Department of Applied Science
First Year B. Tech Academic Year: 2025-26

Lab Project Report

Student Performance Analytics Dashboard

By:

1. Riya Latkar (25070123092)
2. Bhomik Keshi (25070123172)
3. Sara Sagar Potey (25070123102)
4. Sajag Agrawal (25070123100)

Faculty Incharge: Mr. Anurag Mahajan

ACKNOWLEDGEMENT

We are thankful to our Teacher, Mr. Anurag Mahajan, for his continuous support and guidance throughout the completion of this Python and data analytics project. His invaluable input, constructive feedback, and consistent encouragement greatly enhanced both the content and the analytical depth of this project.

We also want to express our sincere gratitude to the college administration for providing the necessary facilities, software tools, and resources required for the successful compilation of this project. Access to Python, Plotly, Pandas, and the Scikit-learn libraries was essential in building the interactive dashboards and visualizations presented here.

We are appreciative of our parents and friends for their unwavering encouragement, moral support, and assistance throughout the project timeline. Their contributions were essential in keeping us motivated and focused, making this project file comprehensive, informative, and visually impactful.

Additionally, we want to acknowledge the online resources, official documentation of Python libraries, and reference books that helped us gather relevant data analysis techniques and visualization best practices. The open-source community around Pandas, Plotly Express, Seaborn, and Scikit-learn deserves special mention for their extensive and well-maintained documentation.

INTRODUCTION

In this Python project, we are analyzing student academic data using the Pandas library. The dataset consists of 50 student records with information on their Department, Attendance, GPA, Living arrangement, and Gender. The project effectively processes, cleans, and analyzes this data to extract meaningful insights.

Essential Python libraries such as Pandas and NumPy are utilized to ensure efficient data manipulation and analysis. Core data operations including shape inspection, null value handling, and descriptive statistics are covered comprehensively.

Various data analysis techniques including categorical analysis, cross-tabulation, grouping, filtering, and sorting are employed to understand student distributions. Data preprocessing steps such as missing value handling, binning, normalization, and encoding are also implemented.

By integrating a variety of data analysis tools and Python functions, this project offers detailed insights into the student dataset, facilitating informed observation and understanding of academic patterns across departments

OBJECTIVE OF THE PROJECT

1. Exploratory Data Analysis: Inspect the student dataset to understand its structure, dimensions, data types, and basic statistics including mean GPA and attendance across all departments.
2. Categorical Data Analysis: Analyze distribution of students by Department, Gender, and Living arrangement. Perform cross-tabulation to understand inter-category relationships and percentage distributions.
3. Missing Value Treatment: Identify and handle missing values in the dataset — particularly GPA — by imputing with mean and median, and replacing placeholders with NaN.
4. Data Binning and Formatting: Categorize continuous attendance data into Low, Medium, and High bins; perform type conversion; string formatting; and rounding.
5. Normalization and Encoding: Apply Min-Max, Z-score, and Decimal Scaling normalization on GPA and Attendance. Implement Label Encoding and One-Hot Encoding for categorical variables.
6. Comprehensive Student Performance Analysis: Analyze and visualize GPA and attendance data across the entire student dataset to identify overall academic trends, high-performing groups, and areas that require improvement.
7. Department-wise GPA Ranking: Compare the average GPA across different academic departments to determine which departments consistently produce top-performing students and which require additional academic interventions.
8. Gender-based Distribution Analysis: Examine the distribution of male and female students within the dataset and explore whether there are any gender-based differences in academic performance metrics such as GPA and attendance.
9. Impact of Living Arrangement on GPA: Investigate whether a student's living arrangement (living at home vs. paying guest/hostel) has a statistically significant effect on their academic performance as measured by GPA.
10. Attendance and GPA Correlation: Determine the strength and direction of the relationship between student attendance rates and GPA scores using scatter plots with OLS trendlines and correlation heatmaps.
11. Interactive Dashboard Development: Design and implement a fully interactive, multi-panel analytics dashboard using Plotly that allows educators and administrators to dynamically filter, explore, and compare student performance metrics.
12. 3D Multi-dimensional Visualization: Create a 3D scatter plot to simultaneously explore the interplay between attendance, GPA, and department encoding, enabling a richer understanding of student clustering patterns.
13. Hierarchical Data Exploration with Sunburst Chart: Build a sunburst chart to represent the nested, hierarchical distribution of students across departments, living arrangements, gender categories, and GPA performance categories.
14. Key Performance Indicator (KPI) Monitoring: Design gauge charts to provide at-a-glance summaries of overall average GPA, average attendance, and the top-performing department's GPA for quick institutional assessment.

RAW DATA

The dataset consists of 50 student records with 6 columns: Student_ID, Department, Attendance, GPA, Living, and Gender. The data covers students from 6 engineering departments.

The dataset used in this project contains student academic records sourced from a university's academic management system. The raw data file is named 'raw data students.xlsx' and is loaded directly into a Pandas DataFrame at the start of the analysis pipeline. The dataset includes a wide range of variables that capture student demographics, academic performance, and behavioral metrics.

Each row in the dataset represents a unique student record, containing the following key columns:

- Student_ID: A unique identifier assigned to each student
- Gender: The gender of the student (Male / Female)
- Department: The academic department the student is enrolled in
- GPA: The student's Grade Point Average on a scale of 0 to 10
- Attendance: The student's class attendance percentage (0 to 100)
- Living: The student's living arrangement (Home / PG - Paying Guest)

Department	No. of Students
AIML	9
Robotics	9
Mechanical	8
ENTC	8
Civil	8
CS	8

The dataset has 2 missing values in the GPA column (Student10 and Student17). All other columns are complete. The overall average GPA is 8.01 and average attendance is 79.94%.

Summary Statistics:

Statistic	Attendance	GPA
Count	50	48
Mean	79.94	8.01
Min	60	6.30
25th Percentile	72.00	7.00
Median	80.50	7.95
75th Percentile	88.00	8.90
Max	97	10.00

Before performing any analysis, the data undergoes a thorough validation and cleaning process. This includes checking for and removing rows with missing values in critical columns (GPA and Attendance), filtering out records where GPA falls outside the valid range of 0 to 10, and removing records where Attendance falls outside the valid range of 0 to 100. This ensures all subsequent visualizations and statistical computations are based only on clean, reliable data.

LIBRARIES AND FUNCTIONS USED

This project utilizes several well-established Python libraries, each serving a specific role in the data analysis and visualization pipeline. The following sections describe each library and its purpose within the project.

1. Pandas

Pandas is the foundational data manipulation library in Python. In this project, Pandas is used to load the raw Excel dataset using pd.read_excel(), inspect the data for missing values using df.isnull().sum(), filter and clean the data by dropping invalid rows, create new derived columns such as GPA_Category using pd.cut(), and compute grouped statistics such as department-wise mean GPA using df.groupby(). Pandas DataFrames serve as the central data structure throughout the entire project pipeline.

2. Matplotlib and Seaborn

Matplotlib provides the foundational plotting infrastructure for Python. In this project, it is used to configure the visual styling of static charts via plt.style.use('seaborn-v0_8-darkgrid'). Seaborn, which is built on top of Matplotlib, provides additional aesthetic improvements through sns.set_palette('husl'). While the primary interactive charts are built with Plotly, Matplotlib and Seaborn serve as the base styling layer.

3. Scikit-learn (sklearn)

Scikit-learn's LabelEncoder from sklearn.preprocessing is used to transform the categorical Department column into numeric integer codes. This encoding is essential for plotting department data on numeric axes, specifically in the 3D scatter plot where all three axes require numeric values. LabelEncoder maps each unique department name to a unique integer in a sorted order.

4. Plotly Express (px)

Plotly Express is a high-level charting library that simplifies the creation of interactive, web-based visualizations. In this project, px.scatter() is used to create the Attendance vs. GPA scatter plot with an OLS regression trendline, px.scatter_3d() generates the three-dimensional scatter visualization, px.sunburst() creates the hierarchical student distribution chart, and px.colors.qualitative.Set3 provides a diverse color palette for the department bar charts.

5. Plotly Graph Objects (go)

Plotly Graph Objects provides fine-grained, low-level control over chart construction. In this project, it is used to create bar charts (go.Bar), histograms (go.Histogram), box plots (go.Box),

heatmaps (go.Heatmap), and gauge/indicator charts (go.Indicator). Each trace is carefully configured with hover templates, color schemes, and text annotations for an optimal user experience.

6. Plotly Subplots

The make_subplots() function from plotly.subplots is used to construct the main 2x3 analytics dashboard. This function arranges six different chart types — bar, histogram, scatter, box, and heatmap — into a single cohesive figure with a unified layout, consistent styling, and shared interactive controls such as dropdown menus.

The following operations and techniques were performed:

- Data Inspection: size, shape, info, head, tail, sample, columns
- Missing Value Handling: isnull, isna, dropna, fillna, replace
- Categorical Analysis: value_counts, crosstab, groupby
- Data Filtering and Sorting: Boolean indexing, sort_values
- Data Binning: pd.cut with custom bins and labels
- Normalization: Min-Max, Z-score, Decimal Scaling
- Encoding: Label Encoding, One-Hot Encoding, Dummy Encoding

DATA INSPECTION

Data inspection is the first step of any data analysis pipeline. It involves understanding the structure, size, and basic properties of the dataset before performing any transformations or analysis.

The following functions were used to inspect the dataset:

```
import pandas as pd
import numpy as np
df = pd.read_csv("/content/raw data students.csv")

df.size       # Total number of elements: 300 (50 rows x 6 columns)
df.shape      # (50, 6)
df.info()     # Data types and non-null counts
df.head()     # First 5 rows
df.tail()     # Last 5 rows
df.sample(5)  # Random 5 rows
df.columns    # Column names
df.isnull().sum()   # Missing values: GPA has 2
df.describe()       # Summary statistics
df.duplicated().sum() # 0 duplicates
df.nunique()        # Unique values per column
```

The dataset has 50 rows and 6 columns with no duplicate records. Only the GPA column has 2 missing values. The shape function confirms the dimensions as (50, 6) and info() shows the data types for each column.

CATEGORICAL DATA ANALYSIS

Categorical data analysis involves examining non-numeric variables to understand their distribution and relationships. In this project, the three categorical columns are Department, Gender, and Living.

```
# Count distribution
print(df['Department'].value_counts())
print(df['Living'].value_counts())
print(df['Gender'].value_counts())

# Percentage distributions
print(df['Department'].value_counts(normalize=True)*100)
print(df['Living'].value_counts(normalize=True)*100)
print(df['Gender'].value_counts(normalize=True)*100)

# Cross-tabulation
print(pd.crosstab(df['Department'], df['Gender']))
print(pd.crosstab(df['Department'], df['Living']))
print(pd.crosstab(df['Department'], df['Gender'], normalize='index')*100)
```

The gender distribution is perfectly balanced at 50% Male and 50% Female. Similarly, 50% of students live at Home and 50% in a PG. AIML and Robotics have the highest student counts (9 each), while the remaining departments each have 8 students.

Category	Male	Female
AIML	5	4
Robotics	4	5
Mechanical	4	4
ENTC	3	5
Civil	4	4
CS	5	3

DEALING WITH MISSING VALUES

Missing data is a common issue in real-world datasets. Proper handling of missing values is essential to maintain data integrity and ensure accurate analysis. Python provides several methods through Pandas to detect and treat missing values.

```
# Detect missing values
df.isna()               # Boolean DataFrame
df.notna()              # Inverse boolean DataFrame
df.isnull()             # Same as isna()
df.isna().sum()         # Count missing per column
df.isna().sum(axis=1)   # Count missing per row

# Remove missing rows
df.dropna()

# Fill missing values
df.fillna(value='DEFAULT')          # Fill with constant
df.fillna(0)                         # Fill with zero
df.fillna(df.mean(numeric_only=True)) # Fill with column mean

# Replace placeholder with NaN
df.replace('-', np.nan, inplace=True)

# Convert column to numeric
df['GPA'] = pd.to_numeric(df['GPA'], errors='coerce')

# Impute GPA with mean
df['GPA'].fillna(df['GPA'].mean(), inplace=True)

# Impute GPA with median
df['GPA'].fillna(df['GPA'].median(), inplace=True)
```

The GPA column had 2 missing values (Student10 and Student17). These were imputed using the mean GPA of 8.01. The fillna() function ensures the dataset remains complete for downstream analysis without discarding records.

DATA BINNING AND DATA FORMATTING

Data binning (also called discretization) converts continuous numerical data into categorical groups. Formatting ensures consistent data types and structure throughout the dataset, which is critical for analysis and visualization.

```
# Data Binning - Attendance into categories
bins1 = [0, 70, 85, 100]
labels1 = ['Low', 'Medium', 'High']
df['Attendance_Category'] = pd.cut(df['Attendance'],
                                   bins=bins1, labels=labels1)

# Data Type Conversion
df['Attendance'] = df['Attendance'].astype(float)

# String Formatting - convert to uppercase
df['Living'] = df['Living'].str.upper()

# Rounding Values
df['Attendance'] = df['Attendance'].round(2)

# Unique values and counts
print(df['Department'].unique())
print(df['Department'].value_counts())
```

Attendance is binned into three categories: Low (below 70), Medium (70-85), and High (above 85). Students with High attendance predominantly belong to the ENTC and CS departments. This binning helps identify at-risk students with low attendance.

Attendance Category	Range	No. of Students
Low	60 - 70	15
Medium	71 - 85	20
High	86 - 97	15

DATA NORMALIZATION AND ENCODING

Normalization rescales data to a standard range, making it suitable for machine learning algorithms. Encoding converts categorical variables into numerical representations that models can process.

```
# Min-Max Normalization
df['GPA_MinMax'] = (df['GPA'] - df['GPA'].min()) /
                   (df['GPA'].max() - df['GPA'].min())

# Normalize Multiple Columns
cols = ['Attendance', 'GPA']
df_norm = (df[cols] - df[cols].min()) /
          (df[cols].max() - df[cols].min())

# Z-score Normalization
df['GPA_Zscore'] = (df['GPA'] - df['GPA'].mean()) /
                   df['GPA'].std()

# Decimal Scaling
df['GPA_DS'] = df['GPA'] / 100000

# Label Encoding
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['Department'])

# Dummy Encoding (Drop First)
df_dummy = pd.get_dummies(df, columns=['Department'],
                          drop_first=True)
```

Min-Max Normalization scales GPA values to the range [0, 1]. Z-score normalization standardizes data to have mean=0 and standard deviation=1. One-Hot Encoding creates binary columns for each department value, making it suitable for regression and classification models.

MAIN DASHBOARD (2x3 SUBPLOT LAYOUT)

The centerpiece of the Student Performance Analytics project is a 2x3 interactive dashboard built using Plotly's make_subplots() function. This dashboard consolidates six distinct visualizations into a single, unified figure with a shared layout, consistent color palette, and interactive dropdown controls. The dashboard is rendered at a width of 1400 pixels and height of 800 pixels for optimal desktop viewing.

Chart 1: Gender Distribution (Row 1, Column 1)

A bar chart displaying the count of male and female students in the dataset. Built using go.Bar(), the chart uses distinct colors — coral red (#FF6B6B) for one gender and teal (#4ECDC4) for the other — to visually differentiate the groups. Value labels are displayed directly on each bar using textposition='auto'. The hover template provides the exact count on mouse-over.

Chart 2: Department GPA Ranking (Row 1, Column 2)

A grouped bar chart showing the average GPA for each academic department. Each department is rendered as a separate bar trace using a dynamically assigned color from Plotly's Set3 qualitative color palette. The department-wise mean GPA is pre-computed using Pandas' groupby operation. This chart allows administrators to instantly identify which departments are performing above or below the institutional average.

Chart 3: GPA Distribution Histogram (Row 1, Column 3)

A histogram displaying the frequency distribution of student GPA scores across 15 equal-width bins. Built using go.Histogram(), the chart uses a soft green color (#A8E6CF) with 80% opacity for a clean, readable appearance. This chart reveals the shape of the GPA distribution — whether it is normal, skewed, or bimodal — providing insight into the overall academic health of the student population.

Chart 4: Attendance vs. GPA Scatter Plot (Row 2, Column 1)

A scatter plot with an OLS (Ordinary Least Squares) regression trendline, generated using Plotly Express's trendline='ols' parameter. Points are color-coded by gender, and each point displays the Student_ID on hover. The trendline visually represents the linear relationship between attendance and GPA, supporting or refuting the hypothesis that higher attendance correlates with better academic performance.

Chart 5: GPA by Living Arrangement (Row 2, Column 2)

A box plot comparing the GPA distributions of students living at home versus those in paying guest (PG) accommodations. Built using go.Box(), each box represents the interquartile range (Q1 to Q3) of GPA scores, with the median line inside and whiskers extending to the minimum and maximum valid values. This chart helps determine whether living arrangements significantly affect academic outcomes.

Chart 6: Correlation Heatmap (Row 2, Column 3)

A 2x2 heatmap showing the Pearson correlation coefficients between Attendance and GPA. Built using go.Heatmap() with the RdBu_r colorscale (red for negative, blue for positive correlations), the heatmap uses zmid=0 to center the color scale at zero. The hover template shows the exact correlation value to three decimal places on mouse-over.

DASHBOARD LAYOUT AND INTERACTIVE CONTROLS

Layout Configuration

The overall dashboard layout is configured through fig.update_layout(), which sets the following key properties:

- Title: Centered at the top with bold Arial Black font, size 24, in dark navy color (#2C3E50)
- Background: Plot area uses light gray (#F8F9FA) and the outer paper uses clean white (#FFFFFF) for a modern look
- Dimensions: Width of 1400px and height of 800px for full desktop viewing
- Grid Lines: Both X and Y axes display subtle light gray grid lines (#E9ECEF) for readability
- Hover Mode: Set to 'closest' so that only the nearest data point's tooltip is shown

Dropdown Menu Filter

An interactive dropdown menu is added to the dashboard using the updatemenus parameter. This dropdown provides two filter options:

- All Departments: Makes all chart traces visible simultaneously, showing the complete dashboard
- Only GPA Charts: Toggles visibility to show only the GPA-related charts (Department GPA Ranking, GPA Histogram, GPA by Living, and the Correlation Heatmap), hiding the Gender Distribution and Attendance Scatter charts

This dropdown control is positioned in the top-left corner of the figure and defaults to the 'All Departments' state. It allows users to focus their analysis on specific aspects of student performance without leaving the dashboard.

Hover Interactivity

Each chart trace is configured with a custom hover template (hovertemplate) that provides context-specific information on mouse-over. For example, the Gender bar chart shows the gender label and exact count, the Department bar chart shows the department name and average GPA to two decimal places, the histogram shows the GPA bin range and frequency count, the scatter plot shows attendance, GPA, and Student_ID, and the heatmap shows the correlation value to three decimal places.

Color Strategy

The dashboard uses a carefully curated color strategy to ensure visual clarity and accessibility. Categorical data such as gender and department uses the husl Seaborn palette and Plotly's Set3 qualitative scheme. Continuous data such as the correlation heatmap uses the RdBu_r diverging colorscale. GPA-specific indicators in the gauge charts use green (#00B894) for overall GPA, blue (#0984e3) for attendance, and purple (#6C5CE7) for the top department.

3D SCATTER PLOT

One of the most visually compelling features of this project is the 3D scatter plot, which provides a multi-dimensional view of student performance by simultaneously plotting three variables: Attendance (X-axis), GPA (Y-axis), and Department (Z-axis, encoded as integers).

Implementation

The 3D scatter plot is created using Plotly Express's px.scatter_3d() function. The function maps the following data columns to visual properties:

- X-axis: Attendance — the student's class attendance percentage
- Y-axis: GPA — the student's grade point average
- Z-axis: Department_Enc — the integer-encoded department label
- Color: Gender — male students are shown in coral red (#FF6B6B), female students in teal (#4ECDC4)
- Size: GPA — larger bubbles represent students with higher GPA scores, adding a fourth visual dimension
- Hover Name: Student_ID — hovering over any point displays the student's unique identifier

Purpose and Insights

The 3D scatter plot allows for the identification of multi-variate clusters that would be invisible in standard 2D charts. For example, it becomes possible to observe whether high-GPA, high-attendance students are concentrated in specific departments, whether certain departments show a wider spread of GPA values regardless of attendance, and whether gender clusters appear in different regions of the 3D space.

The chart is rendered at 700px height and 1000px width for optimal 3D interaction. Users can rotate the chart by clicking and dragging, zoom in and out using the scroll wheel, and isolate specific gender groups by clicking the legend items. The full interactivity of Plotly's 3D engine makes this chart a powerful exploratory tool for multi-dimensional student data.

Color Mapping

The color_discrete_map parameter explicitly assigns coral red to Male and teal to Female, ensuring visual consistency with the Gender Distribution bar chart in the main dashboard. This consistent color coding across all charts makes it easy for viewers to maintain context as they switch between different visualizations.

SUNBURST CHART

The Sunburst chart provides a hierarchical, drill-down visualization of the student population, organized across four nested levels: Department, Living Arrangement, Gender, and GPA Category. This chart is particularly effective for revealing proportional relationships across multiple categorical dimensions simultaneously.

Hierarchical Structure

The four-level hierarchy of the Sunburst chart is defined as follows:

Level	Variable	Description
1 (Center)	Department	The outermost ring represents academic departments
2	Living	Each department splits into Home vs. PG living arrangements
3	Gender	Each living category further splits by Male vs. Female
4 (Outer Ring)	GPA_Category	The innermost ring shows Poor, Average, Good, or Excellent GPA tiers

Implementation Details

The data preparation for the Sunburst chart involves a multi-column groupby operation: df.groupby(['Department', 'Living', 'Gender', 'GPA_Category']).size().reset_index(name='count'). This creates a flat summary table with one row per unique combination of the four categorical variables and a count of students in each combination. Rows with zero students are filtered out using sunburst_data = sunburst_data[sunburst_data['count'] > 0] to avoid rendering empty segments.

The Sunburst chart is built using px.sunburst() with the path parameter defining the hierarchy, the values parameter mapping to the student count, and the color parameter using the count variable with the Viridis continuous colorscale — darker purple for fewer students, brighter yellow-green for larger groups.

Interactive Features

The Sunburst chart is fully interactive. Users can click on any outer ring segment to zoom into that branch of the hierarchy, revealing the distribution of students within that subgroup. Clicking the center of the chart navigates back up the hierarchy. Hovering over any segment displays the segment label, parent category, count of students, and percentage of the total. The chart is rendered at 700px height and 800px width.

GAUGE CHARTS / KEY PERFORMANCE INDICATORS

The gauge charts provide at-a-glance summaries of three critical institutional key performance indicators (KPIs): overall average GPA, average attendance rate, and the GPA of the top-performing department. These gauges are designed for quick executive-level consumption, allowing administrators to immediately assess institutional health without diving into detailed charts.

Gauge 1: Overall GPA

The first gauge displays the mean GPA of all students in the dataset. It uses Plotly's go.Indicator with mode='gauge+number+delta', where the delta shows the difference between the mean GPA and the median GPA. This delta comparison helps identify whether the GPA distribution is symmetrical or skewed — a positive delta means the mean is higher than the median, indicating a right skew (a few very high performers pulling the average up).

The gauge axis spans 0 to 10 (the full GPA range), the gauge bar is filled in green (#00B894), and a red threshold marker is placed at the maximum GPA value in the dataset. This marker acts as an aspirational ceiling, showing how close the average is to the highest recorded GPA.

Gauge 2: Average Attendance

The second gauge displays the mean attendance percentage across all students. Using mode='gauge+number', it shows the raw average value without a delta comparison. The gauge axis spans 0 to 100 (the full attendance range), and the bar is filled in blue (#0984e3). This gauge provides a rapid assessment of overall student engagement with the academic program.

Gauge 3: Top Department GPA

The third gauge highlights the highest-performing department by displaying its mean GPA. The top department and its GPA are determined dynamically using: top_dept = dept_means.loc[dept_means['GPA'].idxmax()]. The gauge title updates automatically to show the name of the top department. The bar is filled in purple (#6C5CE7), and the axis spans 0 to 10. This gauge celebrates academic excellence at the departmental level.

Layout

All three gauges are arranged side by side in a single row using make_subplots() with 1 row and 3 columns, each cell configured with type='indicator'. The overall layout is set to 400px height with the title 'Key Performance Indicators'. This compact format makes the gauge panel ideal as a summary header for any institutional performance report.

KEY PYTHON FUNCTIONS AND METHODS

This section provides a detailed reference for the core Python functions and methods used throughout the Student Performance Analytics project.

1. df.dropna(subset=[...])

Removes rows from the DataFrame where any of the specified columns contain missing values. In this project, it is applied with subset=['GPA', 'Attendance'] to ensure only complete records are retained for analysis.

2. pd.cut()

Bins continuous GPA values into discrete categorical intervals. The bins parameter defines the boundary points [0, 7, 8, 9, 10], and the labels parameter assigns human-readable names ('Poor', 'Average', 'Good', 'Excellent') to each bin. The resulting GPA_Category column is used in the Sunburst chart.

3. LabelEncoder.fit_transform()

From Scikit-learn's preprocessing module, this method learns the unique class labels in a categorical column and transforms them into integer codes in a single step. Applied to the Department column, it produces the Department_Enc column required for the 3D scatter plot.

4. df.groupby().mean().reset_index()

Groups the DataFrame by the Department column and computes the mean GPA for each group. The reset_index() call converts the grouped result back into a standard DataFrame with Department and GPA as regular columns, which is then used to populate the Department GPA Ranking bar chart.

5. make_subplots()

From plotly.subplots, this function creates a multi-panel figure with a specified grid layout (rows, cols), mixed chart types (specified via the specs parameter), subplot titles, and configurable spacing. It returns a Figure object to which individual traces are added using fig.add_trace() with row and col parameters.

6. fig.update_layout()

Applies global layout settings to the entire Plotly figure, including the title, background colors, font properties, dimensions, hover mode, legend visibility, and interactive UI controls such as the

dropdown menu (updatemenus). This single function call controls the overall visual identity of the dashboard.

DATA ANALYSIS

Data analysis involves systematically examining, cleaning, transforming, and modeling data to discover useful information, support conclusions, and enable informed decision-making. In this project, multiple layers of data analysis are performed, ranging from basic descriptive statistics to advanced multi-dimensional visualizations.

Descriptive Statistics

The foundation of the analysis is descriptive statistics computed via Pandas. The groupby('Department')['GPA'].mean() operation produces the average GPA per department, enabling inter-department comparisons. The value_counts() method on the Gender column provides gender distribution counts used in the bar chart. These statistics serve as the primary inputs for the main dashboard visualizations.

Correlation Analysis

The Pearson correlation between Attendance and GPA is computed using the .corr() method on a subset DataFrame: df[['Attendance', 'GPA']].corr(). This produces a 2x2 symmetric matrix of correlation coefficients, where diagonal elements are always 1.0 (perfect self-correlation) and the off-diagonal elements represent the linear relationship strength between the two variables. A value close to +1 indicates a strong positive correlation (higher attendance → higher GPA), while a value near 0 suggests no linear relationship.

OLS Regression Trendline

The Ordinary Least Squares (OLS) trendline in the scatter plot is computed automatically by Plotly Express using the trendline='ols' parameter. This fits a simple linear regression model (GPA = a + b × Attendance) to the data and overlays the best-fit line on the scatter chart. The slope b indicates how much GPA is expected to increase for every one-percentage-point increase in attendance.

Distribution Analysis

The GPA histogram with 15 bins provides a visual representation of the frequency distribution of GPA scores. Analyzing the shape of this distribution — whether normal, left-skewed, right-skewed, or bimodal — provides insight into the academic diversity of the student population. A right-skewed distribution suggests that most students have moderate GPAs with a few high achievers, while a left-skewed distribution indicates a generally high-performing cohort.

Box Plot Analysis

The box plot comparing GPA distributions between Home and PG living arrangements provides a five-number summary (minimum, Q1, median, Q3, maximum) for each group. By comparing the medians and interquartile ranges of the two groups, it is possible to determine whether living arrangement is associated with academic performance differences. Overlapping interquartile ranges suggest no significant difference, while non-overlapping ranges indicate a meaningful performance gap.

ADVANCED VISUALIZATIONS

Beyond the main dashboard, this project implements three additional advanced visualization types that provide richer, more nuanced perspectives on the student performance dataset. These charts leverage Plotly's most sophisticated rendering capabilities to create immersive, interactive data experiences.

3D Scatter Plot (fig_3d)

The 3D scatter plot built with px.scatter_3d() extends the standard 2D attendance-GPA analysis by adding a third axis representing the encoded department. This allows simultaneous visualization of three quantitative dimensions plus two additional encoded dimensions (gender via color and GPA via bubble size). The result is a rich, rotatable point cloud that enables the identification of multi-variate student clusters that would be invisible in any 2D chart.

Key analytical questions answered by the 3D scatter plot include: Are high-GPA students uniformly distributed across departments, or concentrated in specific ones? Do male and female students cluster differently across the attendance-GPA-department space? Does the relationship between attendance and GPA differ across departments? These questions are difficult to answer using standard 2D charts but become visually apparent in the 3D view.

Sunburst Chart (fig_sunburst)

The Sunburst chart built with px.sunburst() is a radially structured hierarchy chart that shows nested categorical relationships. With four levels — Department, Living, Gender, and GPA_Category — it provides a comprehensive breakdown of how students are distributed across all categorical dimensions simultaneously. The size of each segment is proportional to the number of students in that combination, and the Viridis colorscale maps segment size to a continuous color gradient for additional visual encoding.

The Sunburst chart is particularly useful for identifying underrepresented groups (small segments in the outer ring), comparing performance tiers across genders within the same department, and assessing whether the distribution of GPA categories varies between Home and PG students within each department. The interactive drill-down feature allows users to zoom into any branch for a focused sub-analysis.

Gauge / Indicator Charts (fig_gauge)

The three KPI gauge charts built using go.Indicator provide institutional stakeholders with an executive summary view of three critical metrics. Unlike the exploratory charts in the main dashboard, the gauge charts are designed for quick assessment rather than deep analysis. They communicate whether the institution is meeting its academic targets at a glance, without requiring any chart interpretation skills from the viewer.

The use of the delta parameter in the first gauge (comparing mean to median GPA) adds an analytical layer that most basic dashboards do not include. This delta value serves as a quick skewness indicator: a large positive delta (mean significantly above median) suggests that a small number of exceptionally high performers are inflating the average, masking the typical student's performance.

RESULTS AND FINDINGS

The Student Performance Analytics Dashboard produces several key findings from the dataset. The following sections summarize the most important insights derived from each visualization component.

Department-wise GPA Summary

The Department GPA Ranking bar chart reveals significant variation in average academic performance across departments. The chart enables direct comparison of departmental means, identifying both the highest and lowest performing departments. This information is critical for institutional resource allocation decisions.

Attendance-GPA Relationship

The scatter plot with OLS trendline visually confirms or refutes the hypothesis that attendance is positively correlated with GPA. The direction and steepness of the trendline, combined with the correlation coefficient from the heatmap, provide a quantitative measure of this relationship. A positive slope and correlation coefficient above 0.3 would suggest a meaningful relationship between class attendance and academic performance.

GPA Distribution Shape

The GPA histogram provides insight into the overall academic profile of the student cohort. A distribution concentrated between 7 and 9 suggests a moderately high-performing student population with few outliers at the extremes. The histogram's shape also informs decisions about where to set grade cutoffs and intervention thresholds.

Living Arrangement Impact

The box plot comparison of Home vs. PG GPA distributions provides evidence for or against the hypothesis that living arrangements affect academic performance. If the two distributions show similar medians and interquartile ranges, living arrangement is unlikely to be a significant factor. If notable differences exist, it may suggest that students living at home benefit from family support structures, or alternatively, that hostel students are more independent and self-motivated.

KPI Summary Table

KPI Metric	Visualization	Purpose
Overall Average GPA	Gauge Chart 1	Institutional academic performance benchmark
Average Attendance %	Gauge Chart 2	Student engagement and participation level
Top Department GPA	Gauge Chart 3	Departmental excellence recognition
GPA-Attendance Correlation	Heatmap	Strength of attendance-performance relationship
Gender Distribution	Bar Chart	Demographic balance assessment
GPA Category Distribution	Sunburst Chart	Performance tier breakdown by subgroup

CODE WALKTHROUGH

This section provides a structured walkthrough of the complete project code, organized by functional block. Understanding the code structure is essential for replication, modification, and extension of the dashboard.

Block 1: Imports and Setup

The project begins by importing all required libraries: pandas and numpy for data manipulation, matplotlib.pyplot and seaborn for base styling, sklearn.preprocessing.LabelEncoder for categorical encoding, and plotly.express, plotly.graph_objects, and plotly.subplots for interactive visualization. The warnings.filterwarnings('ignore') call suppresses non-critical warning messages to keep the output clean.

Block 2: Data Loading and Validation

The raw Excel file is loaded using pd.read_excel('raw data students.xlsx'). Missing value counts are printed using df.isnull().sum(). Critical rows are dropped and range validations are applied for both GPA (0-10) and Attendance (0-100) using Boolean indexing. The styling is configured with plt.style.use('seaborn-v0_8-darkgrid') and sns.set_palette('husl').

Block 3: Data Preparation

Department-wise mean GPA is computed and stored in dept_means. The GPA_Category column is created using pd.cut(). The Department_Enc column is created using LabelEncoder().fit_transform() only if it doesn't already exist, preventing re-encoding on repeated runs.

Block 4: Main Dashboard Construction

A 2x3 subplot grid is created using make_subplots() with mixed chart types. Six traces are added sequentially: a bar chart for gender, per-department bar charts from a loop, a histogram for GPA, scatter traces from a Plotly Express figure, box plots for living arrangement comparison, and a heatmap for the correlation matrix. The layout is then configured with titles, colors, dimensions, and the dropdown menu.

Block 5: Supplementary Charts

The 3D scatter plot (fig_3d), sunburst chart (fig_sunburst), and gauge charts (fig_gauge) are each created in separate code blocks and displayed using fig.show(). Each supplementary chart is self-contained with its own layout configuration and does not interact with the main dashboard figure.

CHALLENGES AND SOLUTIONS

Challenge 1: Mixed Chart Types in Subplots

Plotly's make_subplots() requires explicit specification of chart types for each cell when mixing different visualization types (e.g., heatmap and scatter in the same figure). The solution was to pass the specs parameter as a 2D list of type dictionaries, mapping each subplot position to its intended chart type ('bar', 'histogram', 'scatter', 'box', 'heatmap').

Challenge 2: OLS Trendline Integration

Plotly Express automatically generates an OLS trendline when trendline='ols' is specified, but the resulting figure contains multiple traces that need to be extracted and added to the subplot. The solution was to iterate over scatter_fig.data and add each trace individually to the subplot using fig.add_trace(trace, row=2, col=1), which correctly handles both the scatter points and the regression line.

Challenge 3: Dynamic Department Colors

With a variable number of departments in the dataset, hardcoding colors would break if the data changes. The solution was to use px.colors.qualitative.Set3 as a color pool and apply colors cyclically using dept_colors[i % len(dept_colors)], ensuring that any number of departments can be displayed without color conflicts.

Challenge 4: Label Encoding Consistency

Running the notebook multiple times without resetting the kernel could cause the LabelEncoder to re-encode the Department column, potentially overwriting existing encodings with different values. The solution was to wrap the encoding block in a conditional check (if 'Department_Enc' not in df.columns), ensuring the encoding is only performed once per session.

Challenge 5: Sunburst Empty Segments

The groupby operation for the Sunburst chart can produce rows with a count of zero for combinations that don't exist in the data. Passing these zero-count rows to px.sunburst() creates invisible segments that still consume space in the hierarchy, causing visual artifacts. The solution was to filter out zero-count rows using sunburst_data = sunburst_data[sunburst_data['count'] > 0] before passing the data to the Sunburst function.

CONCLUSION

The analysis of the student dataset using Python and Pandas has yielded the following key insights:

Department	Avg Attendance (%)	Avg GPA
AIML	72.8	7.4
Robotics	83.7	8.4
ENTC	93.5	9.6
CS	87.2	8.9
Mechanical	71.8	7.1
Civil	72.4	7.2

With this project, we covered the fundamentals of data analysis in Python and demonstrated the power of the Pandas library in extracting insights from student datasets. Python offers robust tools to efficiently handle and analyze data, from loading CSV files to applying normalization and encoding.

By this analysis, we can conclude that:

1. ENTC students have the highest average attendance (93.5%) and GPA (9.6), indicating consistent academic performance.
2. Mechanical and Civil departments have lower attendance and GPA, which may require academic intervention.
3. Gender and Living arrangement are perfectly balanced (50-50), suggesting no sampling bias in the dataset.
4. Students living in PG generally belong to higher-performance departments (ENTC, CS), while Home students are more spread across departments.

Python's data analysis capabilities, combined with the DATA ANALYSIS techniques of normalization, encoding, and binning, allow us to efficiently prepare this dataset for advanced machine learning applications.

REFERENCES

The following references and resources were consulted during the development and documentation of the Student Performance Analytics Dashboard project:

Python Libraries — Official Documentation

- Pandas Documentation: https://pandas.pydata.org/docs/ — For DataFrame operations, groupby, pd.cut(), and data cleaning methods
- Plotly Python Documentation: https://plotly.com/python/ — For interactive chart creation including go.Bar, go.Scatter, go.Heatmap, px.scatter_3d, and px.sunburst
- Plotly Subplots Guide: https://plotly.com/python/subplots/ — For multi-panel dashboard construction using make_subplots()
- Scikit-learn Documentation: https://scikit-learn.org/stable/ — For LabelEncoder usage in preprocessing
- Matplotlib Documentation: https://matplotlib.org/stable/ — For style configuration and base plot settings
- Seaborn Documentation: https://seaborn.pydata.org/ — For statistical visualization styling

Data Science and Analytics References

- McKinney, W. (2022). Python for Data Analysis, 3rd Edition. O'Reilly Media — Comprehensive reference for Pandas-based data manipulation
- VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media — Reference for NumPy, Pandas, Matplotlib, and Scikit-learn workflows
- Healy, K. (2018). Data Visualization: A Practical Introduction. Princeton University Press — Principles of effective data visualization design

Academic and Institutional References

- Symbiosis Institute of Technology, Pune — Lab Project Guidelines, Academic Year 2024-25
- Department of Applied Science, SIT Pune — First Year B.Tech curriculum materials
- Mr. Anurag Mahajan — Faculty Incharge, Department of Applied Science, SIT Pune

Online Learning Platforms

- Coursera: Data Science with Python Specialization (IBM) — Foundation concepts in Python data analysis

- Kaggle Learn: Data Visualization with Plotly — Practical tutorials for interactive chart creation
- Real Python (https://realpython.com) — Python tutorials for data manipulation and visualization best practices